Smart Sustainable Clothing Ecosystem

Document Type	Product Requirements Document (PRD)
Project Name	Revastra
Version	v1.0
Sprint	7 Days – April 2026
Program	GlobalLogic Java Track TE 2.0
Evaluation Date	10 April 2026
Team Size	4 Members

Team Members

Rakshita	Sachin	Samridhi	Tushar
Core Logic Lead	Order & Laundry Lead	Marketplace Lead	Integration & UI Lead

1. Project Overview

Revastra is a multi-service, sustainable clothing ecosystem built on a microservice architecture. It brings together laundry, recycling, donation, stitching, and upcycling into a single unified platform — promoting a circular clothing lifecycle while creating economic opportunities for skilled workers.

> *Revastra is a baselined, end-to-end delivery. All features must be functional, tested, and demo-ready on Evaluation Day — 10 April 2026.*

FIELD	DETAILS	FIELD	DETAILS
Start Date	3 April 2026	**End Date**	9 April 2026
Evaluation	10 April 2026	**Team Size**	4 Members

2. Problem Statement

The current clothing service landscape suffers from critical gaps:

- Rapid increase in textile waste due to fast fashion and lack of sustainable alternatives
- No trusted, unified platform for clothing services (laundry, tailoring, recycling)
- Skilled individuals (tailors, craftspeople) have no dedicated platform to monetize their abilities
- Safety concerns and lack of verification when booking in-home workers
- No reward or incentive mechanism to encourage clothing donation or recycling

3. Objectives

- Build a complete end-to-end clothing services ecosystem under one platform
- Promote sustainable fashion through reuse, recycling, and upcycling
- Create earning opportunities for skilled workers, students, and homemakers
- Ensure worker trust through phone and admin verification with badge system
- Enable seamless booking, payment, and reward redemption

4. Project Scope

4.1 In Scope

- Laundry services: washing, dry cleaning, ironing, pickup & delivery
- Clothing donation and recycling with reward points
- Stitching, alteration, and fitting services with worker booking
- Upcycle marketplace for skilled workers and students
- Online payment, wallet management, and point redemption
- Worker verification system with admin approval and verified badge
- Ratings and reviews for service providers
- Admin dashboard for user and worker management

4.2 Future Scope

- NGO integration for large-scale clothing donations
- AI-based clothing recommendations and upcycle suggestions
- Real-time GPS tracking for pickup and delivery
- Subscription plans for regular laundry users

5. Core Modules

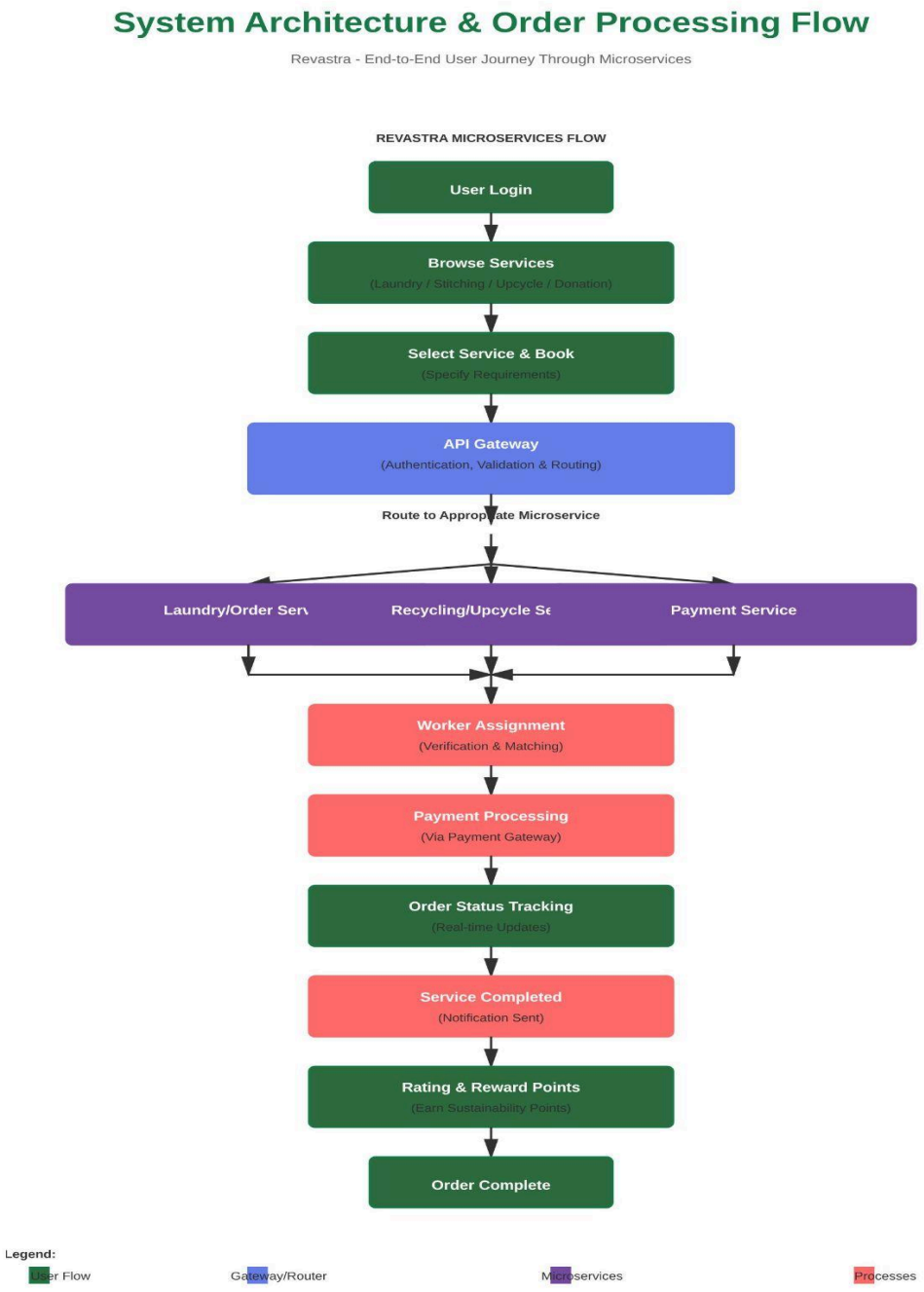
#	Module	Key Features	Owner

1	**Laundry Management**	Washing, dry cleaning, ironing, pickup & delivery	Sachin
2	**Donation & Recycling**	Donate clothes, earn reward points, recycling requests	Rakshita
3	**Stitching & Alteration**	Minor fitting services, worker booking, alteration orders	Samridhi
4	**Upcycle Marketplace**	Worker skill registration, product listing, student/HM earnings	Samridhi
5	**Payment & Wallet**	Online payment gateway, wallet top-up, discount via points	Rakshita
6	**Reward System**	Points earned on donation, redeemable for discounts	Rakshita
7	**Worker Verification**	Phone OTP, admin approval workflow, verified badge display	Samridhi
8	**Ratings & Reviews**	Post-service rating, written review, trust scoring	Tushar
9	**Admin Dashboard**	User management, worker approval, order monitoring	Tushar

6. Microservices Architecture

System Architecture Flow Visualization

The diagram below illustrates the end-to-end request flow through the Revastra microservices architecture, from user login through service completion and reward allocation:



Revastra follows a microservice-based architecture with all mandatory components required by the GlobalLogic sprint evaluation criteria.

COMPONENT	DESCRIPTION	STATUS
API Gateway	Single entry point — routing, auth filter, rate limiting	MANDATORY
Service Registry	Eureka-based service registration and discovery	MANDATORY

User Service	Registration, login, JWT token issue & refresh	MANDATORY
Laundry Service	Booking, pricing, pickup/delivery scheduling	MANDATORY
Recycling Service	Donation intake, reward point calculation and credit	MANDATORY
Upcycle Service	Worker listing, skill registration, product marketplace	MANDATORY
Order Service	Order lifecycle management, status tracking	MANDATORY
Payment Service	Payment gateway integration, wallet management	MANDATORY
Notification Svc	Email/SMS triggers for booking, donation, verification	GOOD TO HAVE

7. Technology Stack

LAYER	TECHNOLOGY	NOTES
Backend	Java 17 + Spring Boot 3.x + REST APIs	Spring Boot 3.2.x recommended
Frontend	React (JSX + Hooks)	Functional components, Axios for API calls
Database	PostgreSQL	JPA/Hibernate ORM
Auth	Spring Security + JWT	Refresh token flow
API Gateway	Spring Cloud Gateway	Route config per microservice
Service Reg.	Netflix Eureka	Auto-registration on startup
Testing	JUnit 5 + Mockito	Min. 70% unit test coverage
Version Control	Git — GitHub	One repo, feature branching
API Testing	Postman	Collection shared in repo
Build Tool	Maven	mvn clean test before every PR

8. User Stories & Acceptance Criteria

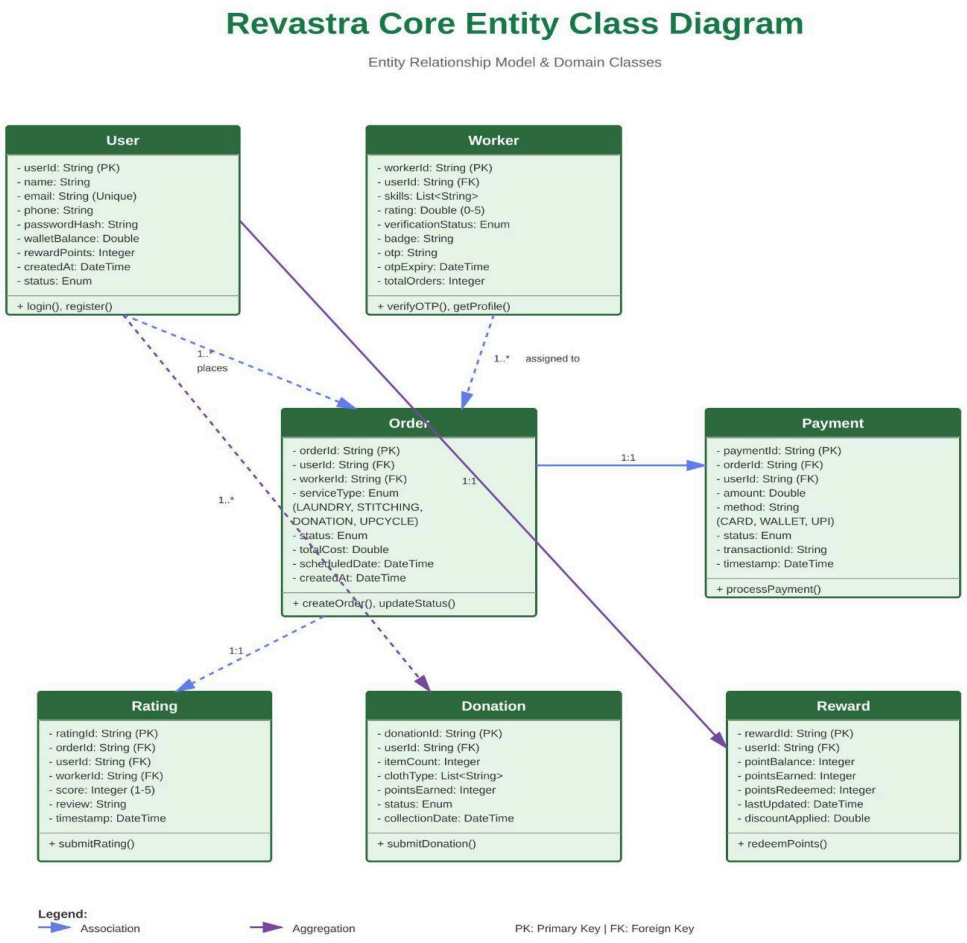
All user stories follow the As a / I want to / So that format with Given-When-Then acceptance criteria as required by the GL Spark evaluation guidelines.

Story ID	US-001
Title	User Registration & Login

As a...	New visitor to the Revastra platform
I want to...	Register using email and password, then log in to access services
So that...	I can book laundry, donate clothes, and track my orders
Acceptance Criteria	1. Given valid email and password, when I submit registration, then my account is created and a JWT is returned.2. Given an already-registered email, when I try to register again, then the system returns a 409 Conflict error.3. Given invalid credentials, when I attempt login, then I receive a 401 Unauthorized response.
Priority	High

Story ID	US-002
Title	Book a Laundry Service
As a...	Registered user
I want to...	Select a laundry service type, choose a pickup slot, and confirm booking
So that...	My clothes are collected, cleaned, and delivered without hassle
Acceptance Criteria	1. Given a logged-in user with a valid address, when they submit a laundry booking, then an order is created with status PENDING.2. Given a selected time slot, when the booking is confirmed, then the user receives a booking confirmation notification.3. Given an invalid or past pickup time, when the user submits, then a validation error is returned.
Priority	High

Story ID	US-003
Title	Donate Clothes & Earn Reward Points
As a...	Registered user with clothes to donate
I want to...	Submit a donation request with item details and earn reward points
So that...	My unused clothes are recycled sustainably and I get points redeemable for discounts
Acceptance Criteria	1. Given a submitted donation request, when it is received, then reward points are credited to the user's wallet.2. Given reward points in the wallet, when the user places an order, then points are applied as a discount.3. Given a donation with invalid item details, when submitted, then the system shows a validation error.
Priority	High



The diagram below represents the core domain entities and their relationships. Each entity corresponds to a database table with primary keys (PK), foreign keys (FK), and relevant business attributes:

Core Entity Class Diagram

Story ID	US-004
Title	Worker Registers & Gets Verified
As a...	Skilled worker (tailor, craftsperson)
I want to...	Register with skills and verify my phone number so I appear in the marketplace
So that...	I can receive bookings and earn income through the platform
Acceptance Criteria	1. Given a valid phone number, when OTP is entered correctly, then phone verification is marked complete.2. Given a verified phone, when admin approves the worker profile, then the Verified badge is displayed on the

	listing.3. Given an unverified or pending worker, when a user searches for workers, then the unverified worker does not appear in results.
Priority	High

Story ID	US-005
Title	Book Stitching or Alteration Service
As a...	Registered user
I want to...	Browse verified tailors and book a stitching or alteration appointment
So that...	I can get my clothes altered by a trusted, verified worker
Acceptance Criteria	1. Given a list of verified workers, when the user selects one and submits a booking, then an order is created.2. Given a booking, when the worker accepts, then the booking status updates to CONFIRMED.3. Given an unverified worker, when a user attempts to book them, then the system blocks the booking.
Priority	Medium

Story ID	US-006
Title	Make Payment & Use Wallet
As a...	Registered user completing an order
I want to...	Pay for services online and use wallet balance or reward points at checkout
So that...	I have a flexible, secure payment experience with discount options
Acceptance Criteria	1. Given a confirmed order, when the user initiates payment, then the payment is processed and order status updates to PAID.2. Given wallet balance, when selected at checkout, then the balance is deducted from the order total.3. Given a failed payment, when retried, then the previous failed transaction is voided and a new one is created.
Priority	High

9. Database Schema (High-Level)

PostgreSQL is used across all services. Each microservice owns its own schema. The following entities are the primary domain objects.

ENTITY	SERVICE	KEY FIELDS
User	User Service	id (PK), name, email, password_hash, role, created_at
Role	User Service	id (PK), role_name (USER / WORKER / ADMIN)
Worker	Upcycle Service	id (PK), user_id (FK), skills, verified, badge_status, rating

Order	Order Service	id (PK), user_id (FK), service_type, status, created_at, total_amount
Laundry	Laundry Service	id (PK), order_id (FK), service_items, pickup_slot, delivery_slot
Recycling	Recycling Service	id (PK), user_id (FK), item_count, description, points_earned
Payment	Payment Service	id (PK), order_id (FK), amount, status, method, transaction_ref
Wallet	Payment Service	id (PK), user_id (FK), balance, reward_points, last_updated
Review	Order Service	id (PK), order_id (FK), rating (1–5), comment, created_at

10. API Overview

All APIs follow REST conventions. The API Gateway routes requests to the correct microservice. All endpoints require a valid JWT Bearer token unless marked public.

METHOD	ENDPOINT	DESCRIPTION	SERVICE
POST	/api/users/register	Register a new user	User Service
POST	/api/users/login	Authenticate and get JWT (public)	User Service
POST	/api/orders	Create a new service order	Order Service
GET	/api/orders/{id}	Fetch order details by ID	Order Service
POST	/api/laundry/book	Book a laundry pickup slot	Laundry Service
GET	/api/laundry/slots	Get available pickup slots	Laundry Service
POST	/api/recycling/donate	Submit a clothing donation	Recycling Service
GET	/api/recycling/points	Get user reward points	Recycling Service
GET	/api/workers	List verified workers	Upcycle Service
POST	/api/workers/register	Worker self-registration	Upcycle Service
POST	/api/payment/pay	Process payment for an order	Payment Service
GET	/api/payment/wallet	Get wallet balance and history	Payment Service
POST	/api/reviews	Submit a review for a service	Order Service

GET	/api/admin/workers/pending	List workers pending approval (admin)	User Service

11. Team Structure & Responsibilities

All members contribute to both Backend (Spring Boot) and Frontend (React). Ownership below indicates the primary responsible developer for each module.

MEMBER	ROLE	BACKEND OWNERSHIP	FRONTEND OWNERSHIP
Rakshita	Core Logic Lead	Recycling Service, Payment Service, Reward System	Donation UI, Wallet UI, Points Dashboard
Sachin	Order & Laundry Lead	Laundry Service, Order Service	Laundry Booking UI, Order Tracking UI
Samridhi	Marketplace Lead	Upcycle Service, Worker Verification	Worker Listing, Stitching & Alteration UI
Tushar	Integration & UI Lead	User Service, API Gateway Integration, Admin Endpoints	Main Dashboard, Navigation, Ratings UI

12. Git Workflow & Repository Guidelines

12.1 Repository

- Repo name format: gl-spark-revastra
- Add all team members as collaborators with write access
- Add GL evaluator as read-only collaborator before 2 April EOD
- Initialize with a README.md describing project setup and run steps

12.2 Branching Strategy

BRANCH	PURPOSE	WHO MERGES
main	Production-ready, stable code only	Team Lead via PR
develop	Integration branch for all features	Team Lead
feature/<story-id>	One branch per user story (e.g. feature/US-001)	Developer via PR
hotfix/<issue>	Critical bug fixes only	Team Lead

RULE: Never commit directly to main. All merges must go through a Pull Request reviewed by at least one other team member.

12.3 Commit Message Convention

PREFIX	MEANING	EXAMPLE
feat:	New feature or user story	feat(US-002): add laundry booking endpoint
fix:	Bug fix	fix(auth): resolve JWT expiry issue
test:	Add or update tests	test(US-003): add JUnit tests for recycling service
docs:	Documentation updates	docs: update README with run steps
chore:	Dependency / config changes	chore: upgrade Spring Boot to 3.2.4
refactor:	Code restructure (no logic)	refactor: extract WorkerValidator class

13. Sprint Timeline

DAY	DATE	MILESTONE	OWNER
1	3 Apr	Project kickoff, PRD finalized, repo created, design slides submitted	All
2	4 Apr	Microservice scaffolding, DB schema creation, API contracts defined	All
3–4	5–6 Apr	Backend service implementation — all core endpoints coded and tested	All
5–6	7–8 Apr	Frontend UI built and connected to backend APIs via API Gateway	All
7	9 Apr	End-to-end testing, bug fixes, code freeze, final push to main	All
Eval	10 Apr	Live demo — all features must be functional with no crashes	All

14. Evaluation Criteria

#	EVALUATION AREA	WEIGHT	KEY EXPECTATIONS
1	**Working Code Demo**	**25%**	All features functional, no crashes during live demo
2	**Architecture & Design**	**20%**	Microservices, API gateway, clear service separation
3	**User Stories & PRD**	**15%**	6 stories with acceptance criteria and traceability
4	**Testing (TDD / JUnit)**	**15%**	Test-first approach, 70%+ coverage, Mockito usage
5	**Code Quality & CVS**	**10%**	Clean commits, branching strategy, no secrets in repo

6	**Diagrams & Slides**	**10%**	All 7 diagram types complete and accurate
7	**Team Collaboration**	**5%**	Equal commits, PR history, peer code reviews visible

15. Unique Selling Points & Conclusion

USP	HOW REVASTRA DELIVERS IT
Circular clothing ecosystem	One app for laundry, donation, recycling, stitching, and upcycling
Verified, trusted workers	Phone OTP + admin approval + visible verified badge
Reward-based sustainability	Earn points for every donation, redeem at checkout
Employment generation	Skilled workers, students, and homemakers earn through the marketplace
Multi-service platform	No need for separate apps — all clothing needs in one place

Revastra is a scalable, socially impactful platform that merges sustainability, convenience, and economic empowerment into a single microservice-based ecosystem. Built on Spring Boot, React, and PostgreSQL, it delivers a complete circular clothing lifecycle — from laundry to donation to upcycling — all within the GL Spark 7-day sprint. Every module is traceable to a user story, backed by tests, and integrated through a clean API gateway architecture.

All team members must have visible Git contributions. Evaluators will assess individual commit history and PR participation. Bring laptops charged — a live demo is expected on 10 April 2026.